

Homework 4

Jake Mc Leroth

2025-04-18

```
library(foreign)
library(mldr)
library(utiml)
library(corrplot)
library(randomForest)
library(e1071)
library(xgboost)
library(ggplot2)
library(reshape2)
```

*In this assignment, the Scene data set is analysed. This is a multilabel data set from image data, with each image having binary labels for Beach, Sunset, Fall Foliage, Field, Mountain and Urban. The data set was first introduced by Boutell et al. (2004). The features for the data were obtained as follows: The RGB colours are converted to the Luv space. Each image was divided into a 7x7 grid of 49 squares, of which each has 3 bands corresponding to a low resolution image and texture features. The mean and variance is calculated for each band, resulting in $49 * 3 * 2 = 294$ features. The goal is to use these features to classify whether each image has a Beach, a Sunset, Fall Foliage, a Field, a Mountain or Urban.*

```
# Import data
data <-
foreign::read.arff("/Users/jakemcleroth/Desktop/University/Masters/Modules/Semester 1/SLT/homework/hw_4/scene_dat")
label_indices <- 295:300
data[label_indices] <- lapply(data[label_indices], function(x)
as.numeric(as.character(x)))

# Create an mldr object
data_mldr <- mldr_from_dataframe(data, labelIndices = label_indices, name =
"scene")
```

Exploratory Data Analysis

An exploratory data analysis is done on the data set:

```
# Missing values
sum(colSums(is.na(data)))

## [1] 0

# Number of columns
data_mldr$measures$num.attributes
```

```
## [1] 300

# Number of observations
data_mldr$measures$num.instances

## [1] 2407

# Number of features
data_mldr$measures$num.inputs

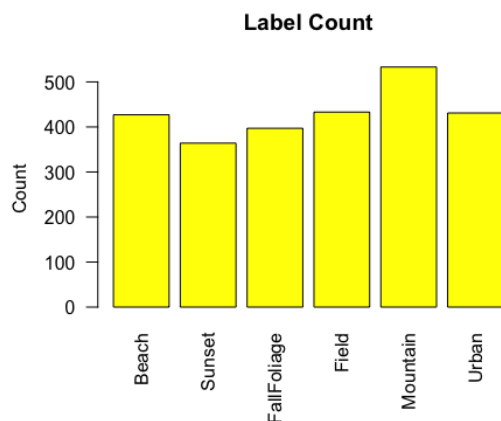
## [1] 294

# Number of labels
data_mldr$measures$num.labels

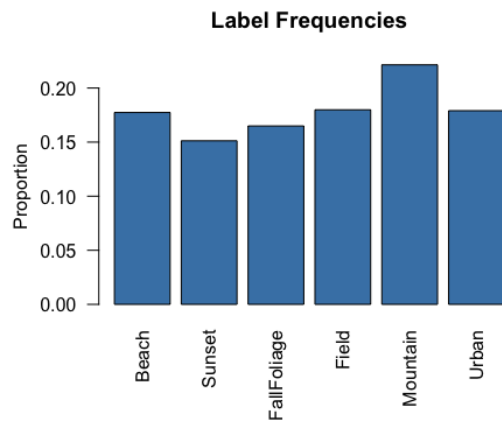
## [1] 6
```

We see there are no missing values to deal with, and confirm there are 6 labels and 294 features. There are 2407 observation (images) in the data set. The labels are looked at in more depth, first looking at the individual labels:

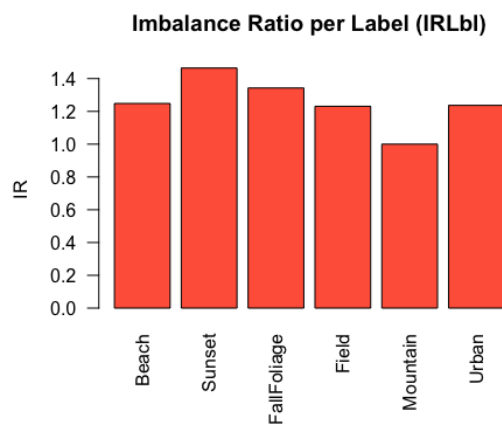
```
# Bar plot for label counts
label_count <- data_mldr$labels$count
barplot(label_count,
        main = "Label Count",
        ylab = "Count",
        las = 2, col = "yellow", names.arg = c("Beach", "Sunset",
"FallFoliage", "Field", "Mountain", "Urban"))
```



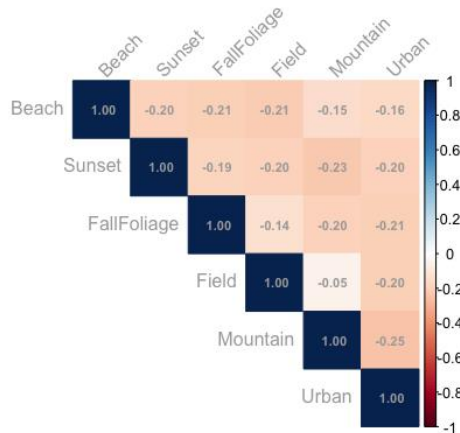
```
# Bar plot for label frequency
label_freq <- data_mldr$labels$freq
barplot(label_freq,
        main = "Label Frequencies",
        ylab = "Proportion",
        las = 2, col = "steelblue", names.arg = c("Beach", "Sunset",
"FallFoliage", "Field", "Mountain", "Urban"))
```



```
# Bar plot for Imbalance Ratio
barplot(data_mldr$labels$IRLbl,
  main = "Imbalance Ratio per Label (IRLbl)",
  ylab = "IR",
  las = 2,
  col = "tomato",
  names.arg = c("Beach", "Sunset", "FallFoliage", "Field", "Mountain",
"Urban"))
```

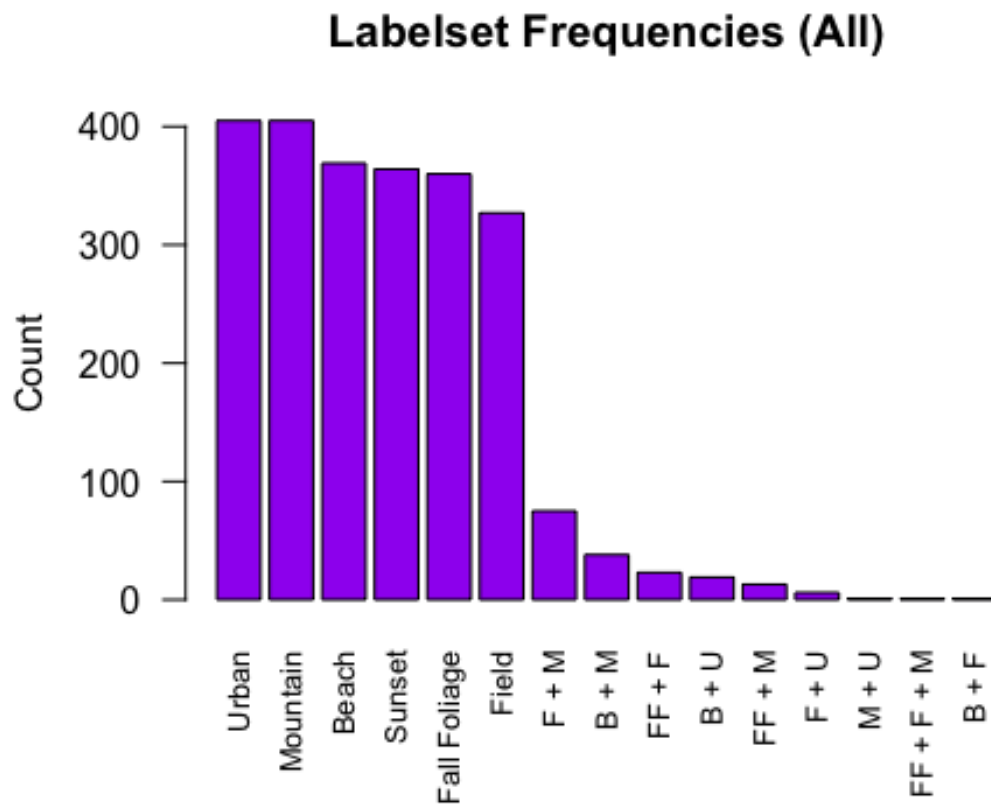


```
# Correlation heat map
label_corr <- cor(data[, label_indices])
corrplot(label_corr,
  method = "color", type = "upper",
  tl.col = "darkgrey", tl.srt = 45,
  addCoef.col = "darkgrey", number.cex = 0.7)
```



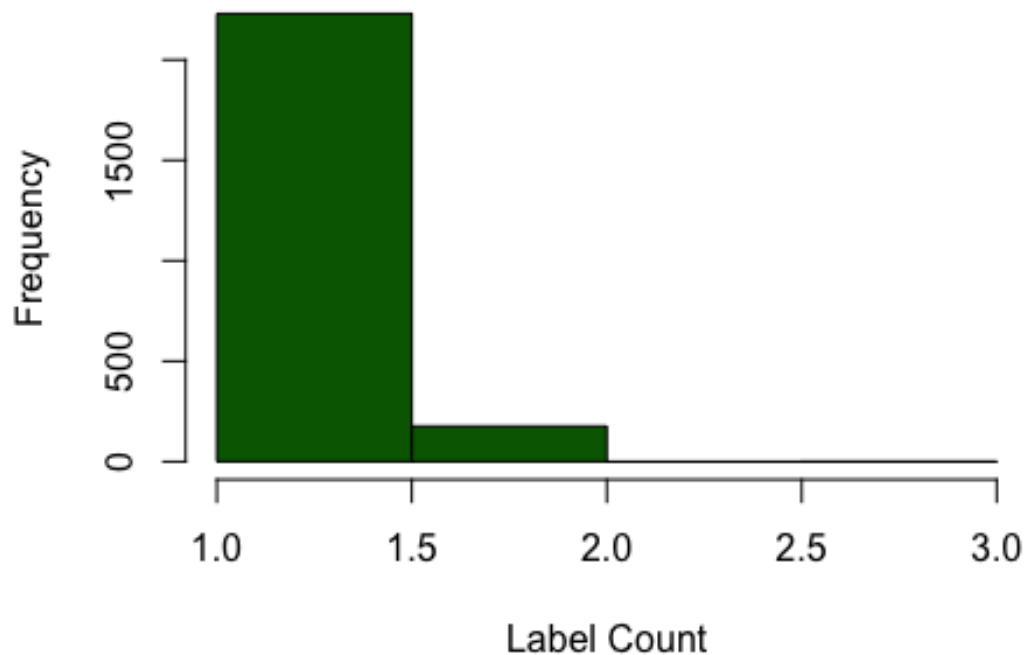
We see from the first three plots that the most common label is Mountain, and the least is Sunset. There is quite a high within-label imbalance, with Mountain having a frequency of 20%. The correlation heat map shows that all labels exhibit a small pairwise negative correlation. We now look at the between label behaviour:

```
# Unique label counts
labelsets <- data_mldr$labelsets
labelset_counts <- sort(labelsets, decreasing = TRUE)
barplot(labelset_counts,
        main = "Labelset Frequencies (All)",
        ylab = "Count",
        las = 2,
        col = "purple",
        names.arg = c("Urban", "Mountain", "Beach", "Sunset", "Fall Foliage",
"Field", "F + M", "B + M", "FF + F", "B + U", "FF + M", "F + U", "M + U", "FF
+ F + M", "B + F"), cex.names = 0.8) # names(labelset_counts)
```



```
# Labels per observation
labels_per_instance <- rowSums(data[, label_indices])
hist(labels_per_instance,
      breaks = max(labels_per_instance),
      main = "Number of Labels per Observation",
      xlab = "Label Count", col = "darkgreen")
```

Number of Labels per Observation



```
# Cardinality
data_mldr$measures$cardinality
```

```
## [1] 1.073951
```

```
# Density
data_mldr$measures$density
```

```
## [1] 0.1789918
```

```
# Concurrence
concurrenceReport(data_mldr)
```

```
## Dataset scene: Mean SCUMBLE 0.0003365239 with CV 4.250558
```

```
##
```

```
## SCUMBLE mean values by label:
```

```
## # Mountain: 0.001479
```

```
## # Field: 0.0009979
```

```
## # Beach: 0.0005447
```

```
## # FallFoliage: 0.0004253
```

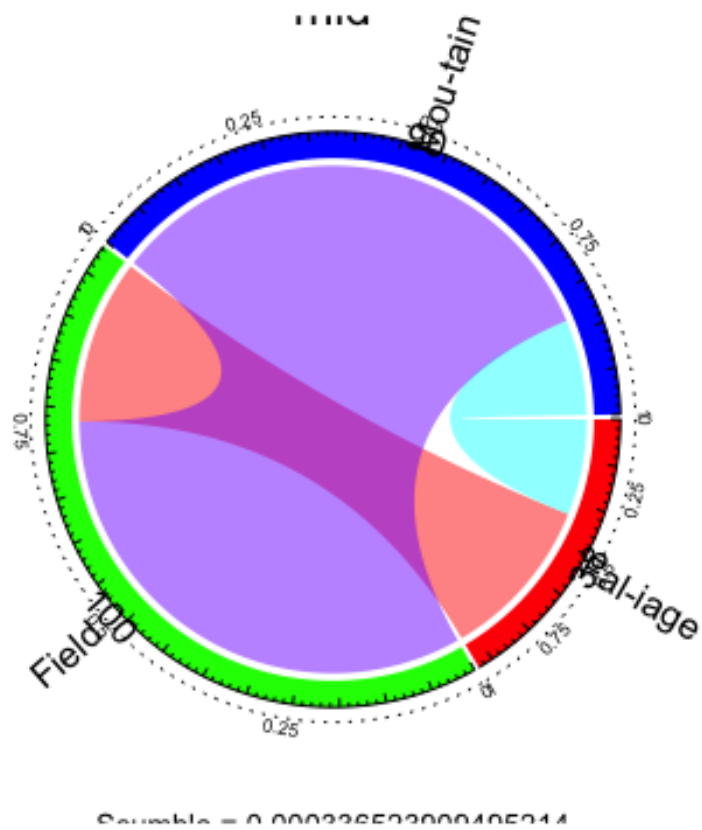
```
## # Urban: 1.354e-05
```

```
## # Sunset: 0
```

```
##
```

```
## Minority label FallFoliage (297, SCUMBLE 0.0004253219) interacts with:
```

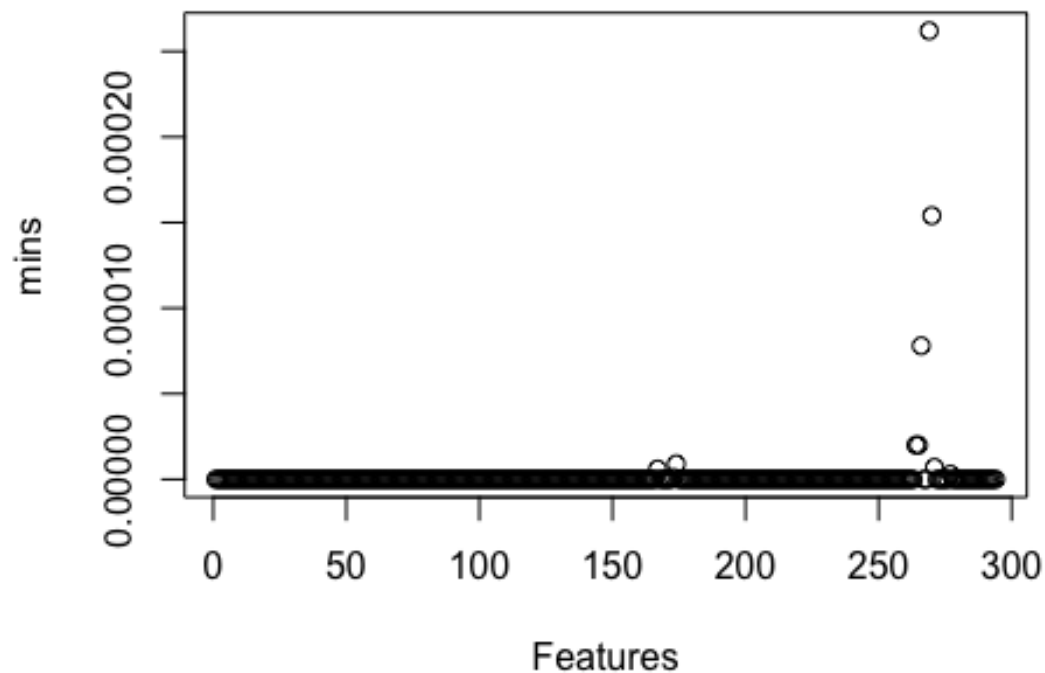
```
## # Field (298, SCUMBLE 0.0009978825): 24 interactions
## # Mountain (299, SCUMBLE 0.001478652): 14 interactions
##
## Minority label Sunset (296, SCUMBLE 0) interacts with:
## # (, SCUMBLE ): NA interactionNA
```



There are 15 unique labelsets, 3 of which are only observed once. Urban and Mountain are the most common labelset, with the rest of the individual features following behind. The max labels present in any labelset is only two. By far the majority of observations only have one label, with the average amount of labels per observation (cardinality) being 1.073951. The density is 0.1789918. This means that not only is there a low number of labels per instance, but only a small proportion of the label space is used. The scumble value is very low, meaning that imbalanced labels don't consistently appear with common ones (Charte et al., 2019). From the concurrence report, we see the labels with the most concurrent interactions are between the minority class Fall Foliage with Field or Mountain. We also note that the class with the least observations, Sunset, does not interact with any other classes. The concurrence of the data set is therefore not a big issue. Now the features are looked at. Since there are 294 features, we look at plots of summary statistics, since analysing each of the 294 features individually is not feasible.

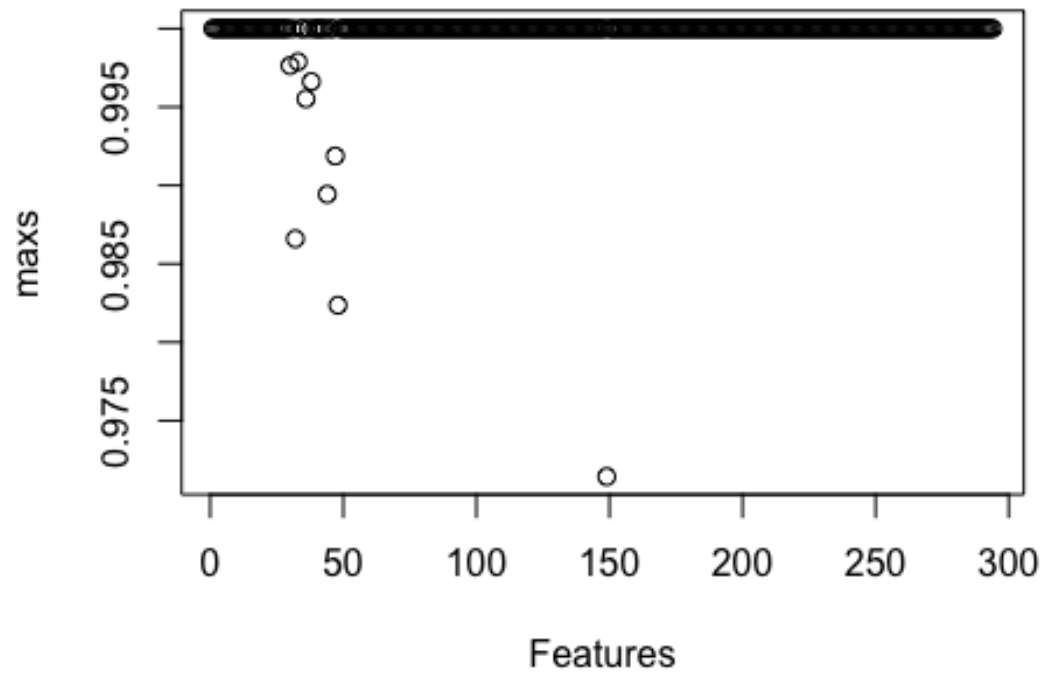
Minimum values

```
mins <- apply(data[, -c(295:300)], 2, min)
plot(mins, xlab = "Features")
```

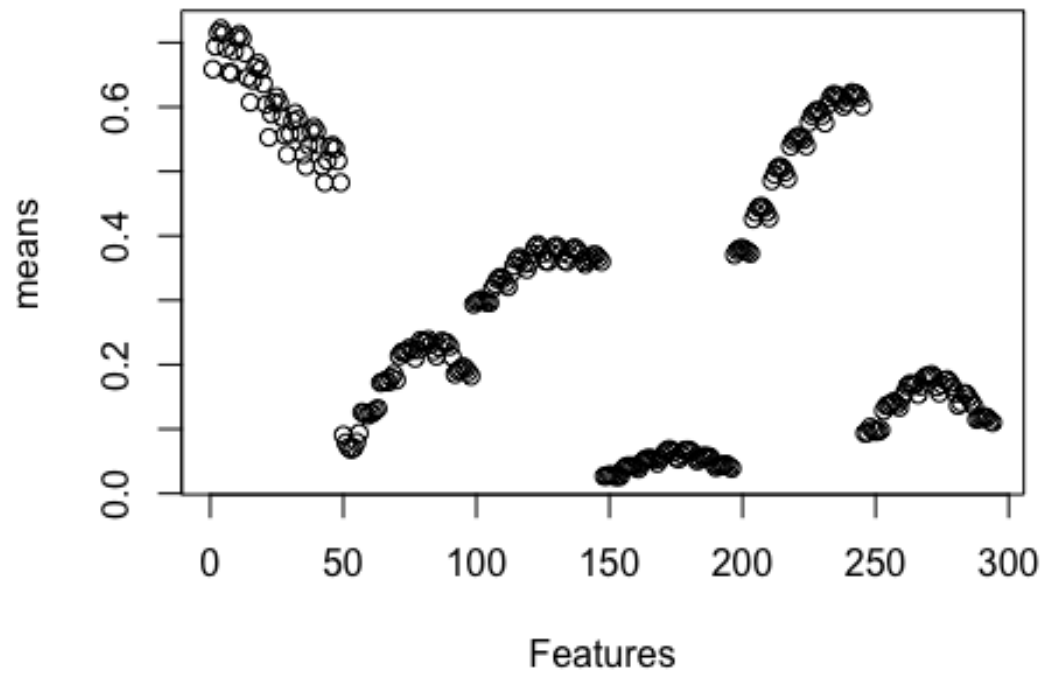


Maximum values

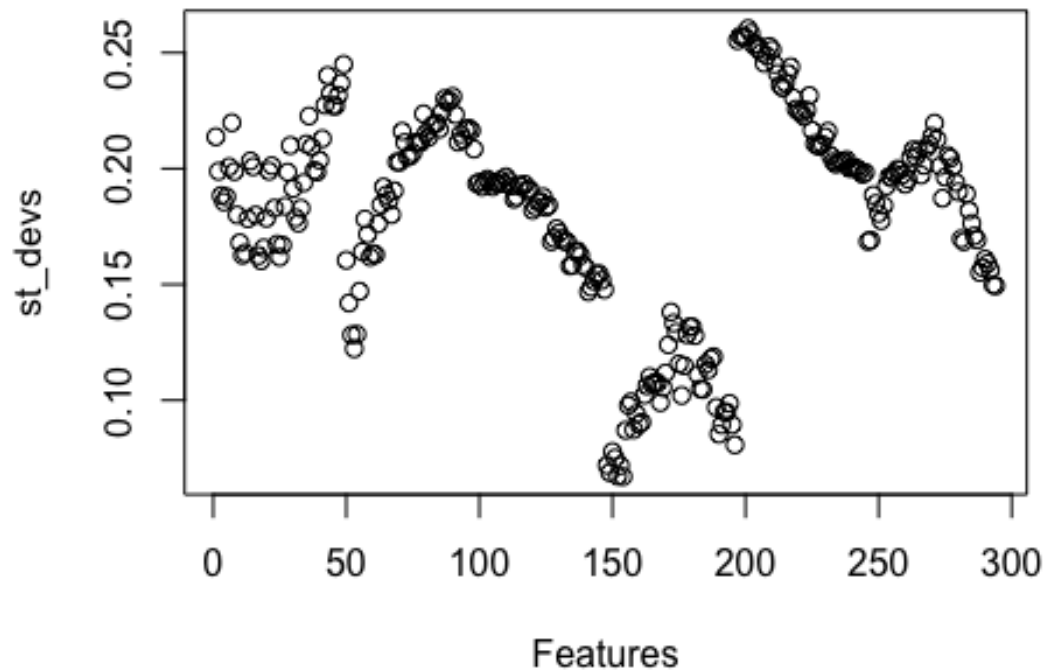
```
maxs <- apply(data[, -c(295:300)], 2, max)
plot(maxs, xlab = "Features")
```



```
# Plot means  
means <- colMeans(data[, -c(295:300)])  
plot(means, xlab = "Features")
```



```
# Plot stdevs  
st_devs <- sqrt(colMeans((sweep(data[, -c(295:300)], 2, means))^2))  
plot(st_devs, xlab = "Features")
```



Looking at the feature data, we see all values range from 0 to 1. The means and standard deviations of the features also exhibits some clustering behaviour.

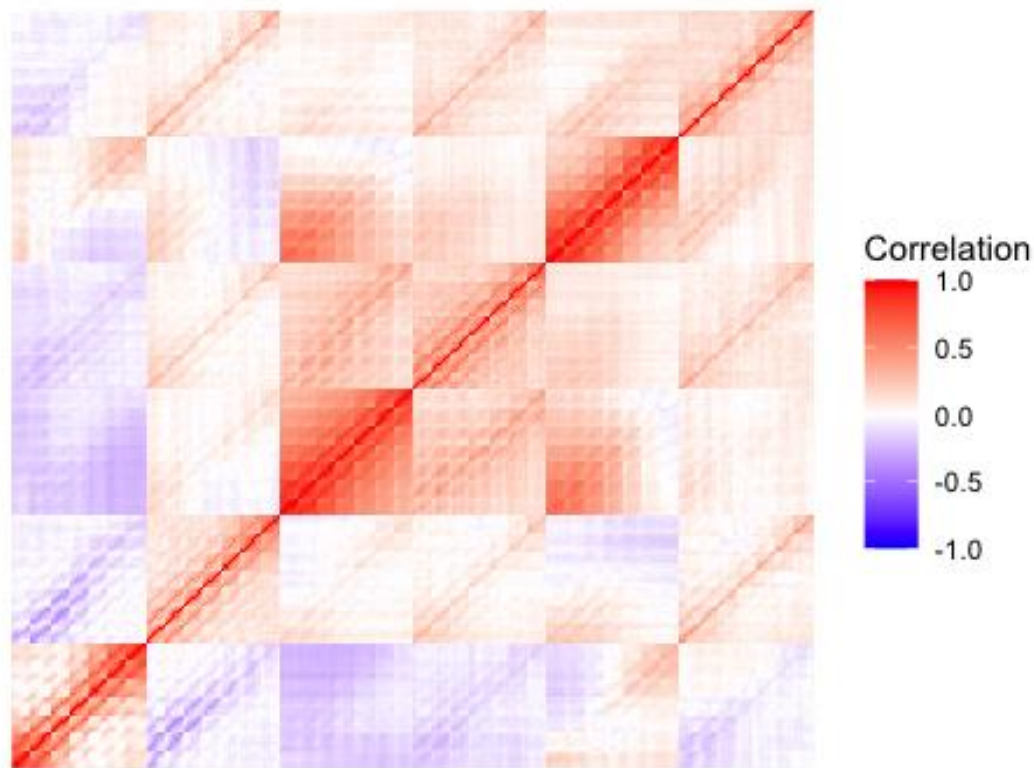
```
# Get correlation matrix
cor_matrix <- cor(data[, -(295:300)])

# Turn matrix into long format for heat map
cor_df <- melt(cor_matrix, varnames = c("Feature1", "Feature2"), value.name =
"Correlation")

# Plot correlation heat map
ggplot(cor_df, aes(x = Feature1, y = Feature2, fill = Correlation)) +
  geom_tile() +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
    midpoint = 0, limit = c(-1, 1), space = "Lab",
    name = "Correlation") +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),
    axis.text.y = element_blank(),
    axis.ticks = element_blank(),
    legend.position = "right")
```

```
) +  
labs(title = "Feature Correlation Heatmap", x = "", y = "")
```

Feature Correlation Heatmap



We see that along the diagonal of the correlation heat map there is some strong correlation among features, however, most of the features seem to have correlations closer to 0.

Model Fitting

Next, we compare several multilabel classification models. The transformation models used are Binary relevance, classifier chains, label powerset and calibrated label ranking. The multi label k-nearest neighbours is used as an adaptation model. For ensemble methods, ensemble binary relevance and ensemble classifier chains are used. For the transformation and ensemble methods, random forests are used as the base model for comparison purposes. The data is split into a training and test set:

```
# train/test split  
set.seed(123) # for reproducibility  
train_indices <- sort(sample(1:nrow(data), size = 1211, replace = FALSE),  
decreasing = F)  
data_train <- data[train_indices,]  
data_test <- data[-train_indices,]  
data_train_mldr <- mldr_from_dataframe(data_train, labelIndices =  
label_indices, name = "scene train")
```

```
data_test_mldr <- mldr_from_dataframe(data_test, labelIndices =  
label_indices, name = "scene test")
```

We fit the models using the utiml package:

Binary relevance with RF

```
br_fit <- br(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_br_predict <- ifelse(predict(br_fit, data_train_mldr)>0.5,1,0)  
test_br_predict <- ifelse(predict(br_fit, data_test_mldr)>0.5,1,0)
```

Classifier Chains with RF

```
cc_fit <- cc(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_cc_predict <- ifelse(predict(cc_fit, data_train_mldr)>0.5,1,0)  
test_cc_predict <- ifelse(predict(cc_fit, data_test_mldr)>0.5,1,0)
```

Label powerset with RF

```
lp_fit <- lp(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_lp_predict <- ifelse(predict(lp_fit, data_train_mldr)>0.5,1,0)  
test_lp_predict <- ifelse(predict(lp_fit, data_test_mldr)>0.5,1,0)
```

Calibrated Label ranking with RF

```
clr_fit <- clr(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_clr_predict <- ifelse(predict(clr_fit, data_train_mldr)>0.5,1,0)  
test_clr_predict <- ifelse(predict(clr_fit, data_test_mldr)>0.5,1,0)
```

Multi-Label knn

```
mlknn_fit <- mlknn(data_train_mldr)  
train_mlknnpredict <- ifelse(predict(mlknn_fit, data_train_mldr)>0.5,1,0)  
test_mlknnpredict <- ifelse(predict(mlknn_fit, data_test_mldr)>0.5,1,0)
```

Ensemble binary relevance with RF

```
ebr_fit <- ebr(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_ebr_predict <- ifelse(predict(ebr_fit, data_train_mldr)>0.5,1,0)  
test_ebr_predict <- ifelse(predict(ebr_fit, data_test_mldr)>0.5,1,0)
```

Ensemble classifier Chains with RF

```
ecc_fit <- ecc(data_train_mldr, base.algorithm = "RF", cores = 6)  
train_ecc_predict <- ifelse(predict(ecc_fit, data_train_mldr)>0.5,1,0)  
test_ecc_predict <- ifelse(predict(ecc_fit, data_test_mldr)>0.5,1,0)
```

The following functions are use to calculate example metrics for the comparison of the model performances:

Example metrics

Hamming Loss

```
hamming_loss <- function(pred, true){  
  return(mean(abs(true - pred)))  
}
```

Accuracy

```
accuracy <- function(pred, true) {
```

```

    return(mean(rowSums(pred & true) / rowSums(pred | true)))
}

# subset accuracy
subset_accuracy <- function(pred, true) {
  return(mean(apply(pred == true, 1, all)))
}

# Precision
precision <- function(pred, true) {
  true_positives <- rowSums(pred & true)
  predicted_positives <- rowSums(pred)
  predicted_positives[predicted_positives == 0] <- 1 # this ensures we don't
  get NaN's i.e. don't divide by 0
  return(mean(true_positives / predicted_positives))
}

# recall
recall <- function(pred, true) {
  true_positives <- rowSums(pred & true)
  possible_positives <- rowSums(true)
  return(mean(true_positives / possible_positives))
}

# f1 score
f1 <- function(pred, true) {
  precision <- precision(pred, true)
  recall <- recall(pred, true)
  return(2 * precision * recall / (precision + recall))
}

```

Results

The models are evaluated as follows, with the results of the training and test sets outputted into tables:

```

# Labels as matrices
train_true_labels <- as.matrix(data_train[, (295:300)])
test_true_labels <- as.matrix(data_test[, (295:300)])

# Lists of metrics and models for output (metrics names correspond to the
# functions)
metrics <- c("accuracy", "subset_accuracy", "hamming_loss", "precision",
"recall", "f1")

train_predictions <- list(
  "binary relevance" = train_br_predict,
  "classifier chains" = train_cc_predict,
  "label powerset" = train_lp_predict,
  "calibrated label ranking" = train_clr_predict,

```



```

##          calibrated label ranking multilabel k-NN
## accuracy          0.553          0.721
## subset_accuracy   0.105          0.672
## hamming_loss      0.149          0.072
## precision         0.553          0.750
## recall            1.000          0.742
## f1                0.712          0.746
##          ensemble binary relevance ensemble classifier chains
## accuracy          0.902          0.901
## subset_accuracy   0.886          0.880
## hamming_loss      0.023          0.025
## precision         0.919          0.922
## recall            0.903          0.901
## f1                0.911          0.912

```

```
print(t(round(test_results, 3)))
```

```

##          binary relevance classifier chains label powerset
## accuracy          0.579          0.599          0.457
## subset_accuracy   0.556          0.578          0.446
## hamming_loss      0.086          0.084          0.105
## precision         0.598          0.617          0.466
## recall            0.583          0.602          0.457
## f1                0.590          0.610          0.461
##          calibrated label ranking multilabel k-NN
## accuracy          0.494          0.656
## subset_accuracy   0.066          0.619
## hamming_loss      0.182          0.094
## precision         0.498          0.676
## recall            0.923          0.675
## f1                0.647          0.675
##          ensemble binary relevance ensemble classifier chains
## accuracy          0.750          0.751
## subset_accuracy   0.721          0.722
## hamming_loss      0.076          0.075
## precision         0.777          0.780
## recall            0.752          0.752
## f1                0.764          0.766

```

On the training set, binary relevance, classifier chains and label powerset all fit the data perfectly, achieving an accuracy of 1 - it seems likely that this will lead to poor out of sample performance due to overfitting. The ensemble methods fit the data training data fairly well, with accuracies and f1 scores above 0.9. The MLkNN fit the training data decently well, with an accuracy of 0.75. The worst fit on the training data was calibrated label ranking, with an accuracy of 0.556 and a sub-set accuracy of 0.112 - by far the lowest. The calibrated label ranking did have a recall of 1, however. Looking at the out of sample performance, ensemble classifier chains performs the best, having the highest accuracy, sub-set accuracy, precision, and f1, and the lowest hamming loss. The only value it doesn't perform the best in is recall, in which calibrated label ranking has the highest. This means the ensemble classifier is the overall best method for classifying the images. We note that the binary relevance, classifier chains and label powerset performance did get significantly worse out of sample, meaning the model did overfit. It is interesting that the calibrated label ranking model has the highest precision, meaning it was the best at avoiding false positives. The second best performing model overall was ensemble binary relevance. This shows the power of ensemble learning, resulting in the ensemble methods being the best overall for classifying images in the data set.

References

- Boutell, M.R., Luo, J., Shen, X. and Brown, C.M. (2004). Learning multi-label scene classification. Pattern recognition, vol. 37, no. 9, pp. 1757–1771.*
- Charte, F., Rivera, A.J., del Jesus, M.J. and Herrera, F. (2019). Dealing with difficult minority labels in imbalanced multilabel data sets. Neurocomputing, vol. 326, pp. 39–53.*